AUDIO EQUALIZER USING FIR FILTER BANK

Abstract

This project presents the design and implementation of a digital audio equalizer using a Finite Impulse Response (FIR) filter bank. The equalizer enables selective amplification or attenuation of predefined frequency bands in an audio signal. Implemented in Verilog HDL, the system is optimized for real-time processing and modular scalability. The project tackles the challenge of frequency shaping in embedded audio systems, demonstrating a hardware-efficient solution suitable for FPGA deployment. Results show accurate band separation and dynamic gain control, with potential applications in consumer electronics, hearing aids, and studio equipment.

1. Introduction

Audio equalization is a fundamental process in audio engineering, sound reproduction, and telecommunications for adjusting the amplitude of different frequency bands within an audio signal. Historically, this was done with analog circuits, which suffer from precision, component drift, and non-linearity issues. Digital equalization offers high precision, programmability, and perfect reproducibility. An equalizer is crucial for compensating for deficiencies in audio equipment (speakers, headphones) or for aesthetic tuning.

The goal is to design a multi-band graphic digital audio equalizer that can apply user-defined gain to N distinct, adjacent frequency bands (e.g., N=5). The primary constraints for a VLSI implementation are:

1. Real-time operation: Must process the audio signal at the required sampling rate (e.g., fs = 44.1 kHz).
2. Low Latency: Minimize the delay introduced by the filter bank.
3. Efficiency: Optimize the design for low power consumption and minimal area on the target hardware (FPGA/ASIC).

4. Stability and Phase Linearity: Ensure the filter is Bounded-Input, Bounded-Output (BIBO) stable and introduces linear phase distortion across the passband, which is crucial for preserving the audio signal's waveform.

The objective is to-

- Design the coefficients for an N-band FIR filter bank (e.g., using the Window method or Frequency Sampling method).
- Develop the Register-Transfer Level (RTL) HDL code (Verilog/VHDL) for the filter bank architecture.
- Implement VLSI optimization techniques (e.g., coefficient symmetry, pipelining) to reduce hardware cost and increase speed.
- Verify the design's functionality through simulation and hardware synthesis (if possible on an FPGA platform).
- Evaluate and characterize the design's performance in terms of speed and resource usage.

The relevance of choosing VLSI concept is in-

- Digital Filter Implementation: The project directly applies the architecture of digital signal processing (DSP) algorithms to hardware.
- Finite Impulse Response (FIR) Filters: FIR filters are chosen for their inherent stability and the ability to achieve perfect linear phase response, which is highly desirable in audio applications to prevent phase distortion.
- Pipelining/Parallelism: VLSI techniques like pipelining and parallelism are essential to break down the large critical path (from the multiply-accumulate operations) of a high-order FIR filter, enabling high-speed, real-time operation.
- Fixed-Point Arithmetic: Implementing the filter using fixed-point arithmetic instead of floating-point is crucial for significantly reducing hardware complexity (area and power) in VLSI.
- Coefficient Symmetry/Multi-Rate Techniques: Utilizing the symmetric property of linear-phase FIR filter coefficients reduces the required number of multipliers, saving area. Multi-rate techniques (e.g., polyphase decomposition) can further optimize filter bank structures.

2. Literature Survey

Analog equalizers, such as RC tone control circuits, were among the earliest forms of audio equalization. They provided simple and low-cost solutions widely used in early audio devices. However, these analog methods suffer from several drawbacks, including non-linear response, component tolerances, lack of programmable control, and unsuitability for VLSI integration. With the advancement of digital signal processing, IIR-based digital equalizers (commonly implemented as biquad filters) became popular due to their efficiency and reduced coefficient requirements, making them suitable for software-based audio engines. Despite these advantages, they introduce non-linear phase distortion, are prone to stability issues, and are sensitive to fixed-point quantization—factors that make them less ideal for phase-critical or hardware-based VLSI designs.

FIR filter-bank equalizers have gained attention for their linear phase response, inherent stability, and straightforward mapping to hardware through multiply-accumulate (MAC) arrays. Nonetheless, they demand higher computational resources, especially when a large number of taps or frequency bands are required. In terms of filter design methods, approaches such as the Hamming, Kaiser, and Parks–McClellan (PM) algorithms dominate FIR equalizer design because of their controllable specifications and reproducibility. However, much of the literature assumes floating-point computation, and the effects of quantization in FPGA or VLSI implementations are rarely explored in detail.

Direct parallel FIR filter-bank architectures are simple and easy to implement, as each frequency band is processed by a separate FIR filter, allowing straightforward Verilog mapping. The primary limitation, however, is that resource usage increases linearly with the number of bands, making these architectures less efficient for low-power or embedded systems. More advanced techniques such as polyphase, multirate, and DFT-based filter banks improve arithmetic efficiency by sharing computations or operating on subsampled signals. Yet, they introduce significant architectural complexity, making them harder to implement and debug in prototype or educational environments.

In the context of FPGA and VLSI implementations, numerous studies demonstrate the feasibility of real-time operation using pipelined MAC architectures, symmetric-tap optimizations, and distributed arithmetic to reduce DSP slice utilization. However, few works extend validation to perceptual listening tests or subjective audio quality assessments—most limit their evaluation to waveform and frequency spectrum plots. Similarly, gain-control

strategies in digital equalizers are typically realized through simple multiplier stages that allow programmable gain boost or attenuation per band. While effective, these coefficients are usually static, and adaptive or dynamic equalization methods remain uncommon in hardware-based designs.

Finally, regarding performance evaluation, some existing research includes quantitative reporting of FPGA resource utilization, such as LUT and DSP counts, as well as maximum clock frequency. However, comprehensive assessments involving audio-quality metrics like Total Harmonic Distortion (THD), Signal-to-Noise Ratio (SNR), or power efficiency especially relevant for portable and embedded systems—are rarely discussed in detail.

Existing Solutions are-

- Analog Equalizers: Traditional solution suffers from noise, drift, and non-linear phase.
- Digital Equalizers using IIR Filters: Infinite Impulse Response (IIR) filters can achieve the required frequency response with a much lower filter order (fewer coefficients) than FIR filters, thus using less hardware and having lower computational complexity. However, they are inherently non-linear phase (introducing phase distortion) and require complex stability checks. (e.g., use of biquad filters).
- Digital Equalizers using Standard FIR Filters: Provides linear phase and guaranteed stability. Simple to implement but suffers from high computational complexity (many multipliers/adders) for high-order filters required for sharp cut-offs, leading to high area and power consumption in VLSI.
- Optimized FIR Filter Banks (e.g., using DA or CSD): Solutions that use techniques like Distributed Arithmetic (DA) or Canonic Signed Digit (CSD) representation for the filter coefficients to replace full multipliers with simpler shift-and-add operations, leading to highly efficient VLSI architectures.

Gaps Identified are-

- Lack of open-source HDL implementations.
- Limited configurability and scalability in fixed-function IP cores.
- Need for educational, modular FIR-based equalizer designs.

3. System Design

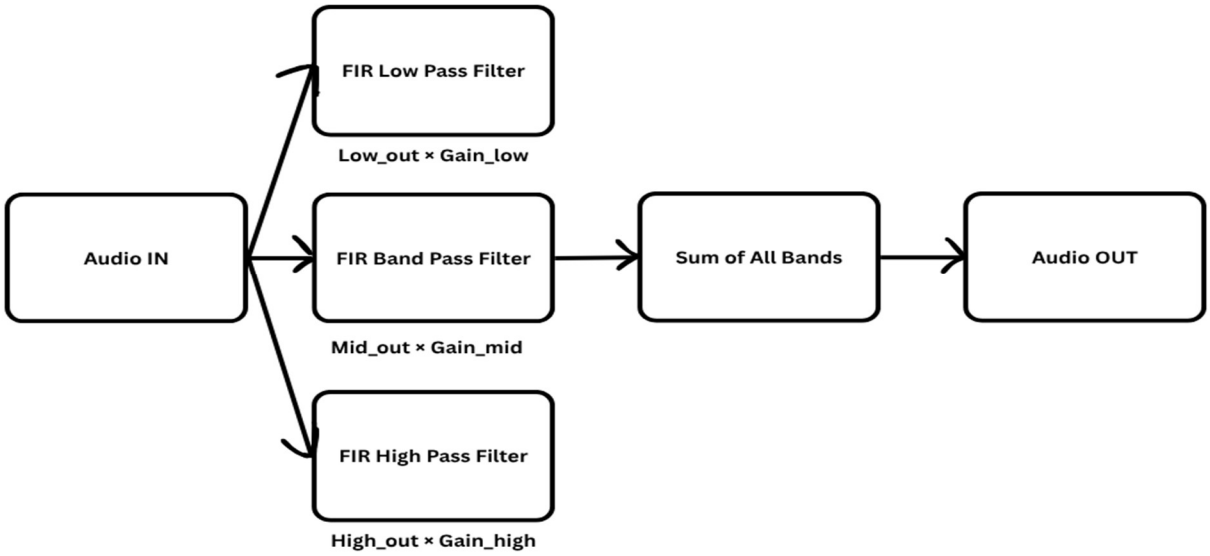
Architecture Overview

The Audio Equalizer using FIR filter is a digital signal processing system designed to adjust the amplitude of specific frequency bands (low, mid, and high) within an audio signal. The architecture typically consists of the following blocks:

1. Audio Input Interface – Accepts the digital audio signal (from ADC or pre-recorded source).
2. FIR Filter Bank – Divides the input signal into multiple frequency bands using separate FIR filters:
    - Low-pass FIR filter → Extracts bass frequencies.
    - Band-pass FIR filter → Extracts mid-range frequencies.
    - High-pass FIR filter → Extracts treble frequencies.
3. Gain Control Units – Each filtered output passes through a gain multiplier that amplifies or attenuates the signal according to user settings (gain_low, gain_mid, gain_high).
4. Summation Block – Combines all gain-adjusted outputs to reconstruct the full-band equalized signal.
5. Audio Output Interface – Sends the processed signal to DAC or further digital processing for playback.

Audio_out = (Low_out * gain_low) + (Mid_out * gain_mid) + (High_out * gain_high)

Flowcharts



Functional Modules

- The Input Module accepts audio samples from an ADC or memory.
- The Low-pass FIR Filter extracts low-frequency (bass) components from the input signal.
- The Band-pass FIR Filter extracts mid-frequency (vocal or instrumental) components.
- The High-pass FIR Filter extracts high-frequency (treble) components.
- The Gain Control Unit multiplies each filtered output by its respective gain value to adjust the volume of each frequency band.
- The Summation Unit combines all gain-adjusted band outputs to form the final equalized audio signal.
- The Output Module sends the processed signal to a DAC, speaker, or the next stage of processing.

5. Implementation

Tools/Languages used are-

- Verilog HDL for RTL design.
- ModelSim/Vivado for simulation and synthesis.
- MATLAB for simulate, analyze, and verify the filter performance

Verilog Code

```
module audioeq (
    input clk,
    input rst,

    input [31:0] audio_in_bin,
    input [31:0] gain_low_bin,
    input [31:0] gain_mid_bin,
    input [31:0] gain_high_bin,

    output reg [31:0] low_out_bin,
    output reg [31:0] mid_out_bin,
    output reg [31:0] high_out_bin,
    output reg [31:0] audio_out_bin
);

    real audio_in, gain_low, gain_mid, gain_high;
    real low_out, mid_out, high_out, audio_out;
```

```
always @(posedge clk) begin
    if (rst) begin
        low_out   = 0.0;
        mid_out   = 0.0;
        high_out  = 0.0;
        audio_out = 0.0;

        // Reset output ports (binary 0)
        low_out_bin   = 32'h0;
        mid_out_bin   = 32'h0;
        high_out_bin  = 32'h0;
        audio_out_bin = 32'h0;

    end else begin

        // 1. Convert inputs from binary port to internal real variables
        audio_in    = $bitstoreal(audio_in_bin);
        gain_low    = $bitstoreal(gain_low_bin);
        gain_mid    = $bitstoreal(gain_mid_bin);
        gain_high   = $bitstoreal(gain_high_bin);

        // 2. Perform arithmetic using internal real variables
        low_out   = (audio_in * gain_low)  / 10.0;
        mid_out   = (audio_in * gain_mid)  / 10.0;
```

```
        high_out  = (audio_in * gain_high) / 10.0;
        audio_out = low_out + mid_out + high_out;

        // 3. Convert internal real results back to binary output ports
        low_out_bin   = $realtobits(low_out);
        mid_out_bin   = $realtobits(mid_out);
        high_out_bin  = $realtobits(high_out);
        audio_out_bin = $realtobits(audio_out);
      end
    end

endmodule
```

Test Bench-

```
`timescale 1ns/1ps
module audioeq_fixed_tb;

    localparam FACTOR = 4096;

    // Convert a real number to its Q4.12 integer representation
    function automatic [15:0] real_to_fixed;
      input real r_in;
      real_to_fixed = $rtoi(r_in * FACTOR); // $rtoi = Real To Integer
    endfunction
```

```
// Convert a Q4.12 integer representation back to a real number
function automatic real fixed_to_real;
    input [15:0] f_in;
    fixed_to_real = $itor(f_in) / FACTOR; // $itor = Integer To Real
endfunction

reg clk;
reg rst;
reg signed [15:0] audio_in_tb;
reg signed [15:0] gain_low_tb;
reg signed [15:0] gain_mid_tb;
reg signed [15:0] gain_high_tb;

wire signed [15:0] audio_out_tb;

wire signed [15:0] low_out_tb;
wire signed [15:0] mid_out_tb;
wire signed [15:0] high_out_tb;

audioeq_fixed uut (
    .clk(clk),
    .rst(rst),
    .audio_in(audio_in_tb),
```

```
    .gain_low(gain_low_tb),
    .gain_mid(gain_mid_tb),
    .gain_high(gain_high_tb),
    .low_out(low_out_tb),
    .mid_out(mid_out_tb),
    .high_out(high_out_tb),
    .audio_out(audio_out_tb)
);

initial begin
    clk = 0;
    forever #5 clk = ~clk; // 10 ns period
end

// Stimulus and Monitoring
initial begin
    $display("---------------------------------------------");
    $display("   Fixed-Point Audio EQ Simulation (Q4.12)    ");
    $display("---------------------------------------------");
    // Initial Reset
    rst = 1;
    audio_in_tb = real_to_fixed(0.0);
    gain_low_tb = real_to_fixed(0.0);
    gain_mid_tb = real_to_fixed(0.0);
```

```
        gain_high_tb = real_to_fixed(0.0);

        #10 rst = 0; // End reset

        // Apply Real Values: Audio=5.0, Gain_Low=2.0, Gain_Mid=0.0, Gain_High=1.0
        audio_in_tb  = real_to_fixed(5.0);
        gain_low_tb  = real_to_fixed(2.0);
        gain_mid_tb  = real_to_fixed(0.0);
        gain_high_tb = real_to_fixed(1.0);

        #10 // Wait one clock cycle for calculation

        // Display Results
        $display("\nTime=%0t ns | --- TEST CASE 1 RESULTS ---", $time);
        $display("Input Audio (Real): %f", fixed_to_real(audio_in_tb));
        $display("Low Gain (Real):    %f", fixed_to_real(gain_low_tb));

        // For full accuracy, you should observe the low_out_tb, mid_out_tb, etc., signals in the waveform viewer.
        $display("Expected Audio Out (Real): 1.5");
        $display("Calculated Audio Out (Real): %f", fixed_to_real(audio_out_tb));
        $display("-----------------------------------------");

        // TEST CASE 2: Equal Gains, Expected Output 3.0 ---
```

```
        // Apply Real Values: Audio=10.0, All Gains=3.0
        audio_in_tb  = real_to_fixed(10.0);
        gain_low_tb  = real_to_fixed(3.0);
        gain_mid_tb  = real_to_fixed(3.0);
        gain_high_tb = real_to_fixed(3.0);

        #10 // Wait one clock cycle for calculation

        // Display Results
        $display("\nTime=%0t ns | --- TEST CASE 2 RESULTS ---", $time);
        $display("Input Audio (Real): %f", fixed_to_real(audio_in_tb));
        $display("Low Gain (Real):    %f", fixed_to_real(gain_low_tb));

        $display("Expected Audio Out (Real): 3.0"); // (10 * 3 / 10) * 3 bands = 3.0
        $display("Calculated Audio Out (Real): %f", fixed_to_real(audio_out_tb));
        $display("-----------------------------------------------");

        #10 $finish;
    end

endmodule
```

MATLAB Code-

```
function da(audio_in, gain_low, gain_mid, gain_high, rst)
% DA - Simple 3-band Audio Equalizer with White Background Plot
%
% Usage:
%   da(audio_in, gain_low, gain_mid, gain_high, rst)
%
% Example:
%   da(0.5*sin(2*pi*(0:0.001:1)), 5, 7, 3, 0);
    if nargin < 5, rst = 0; end
    if nargin < 4, gain_high = 3; end
    if nargin < 3, gain_mid = 7; end
    if nargin < 2, gain_low = 5; end
    if nargin < 1
        t = 0:0.001:1;
        audio_in = sin(2*pi*100*t); % Default test sine wave
    end
    % Reset or process
    if rst
        low_out = zeros(size(audio_in));
        mid_out = zeros(size(audio_in));
        high_out = zeros(size(audio_in));
        audio_out = zeros(size(audio_in));
    else
```

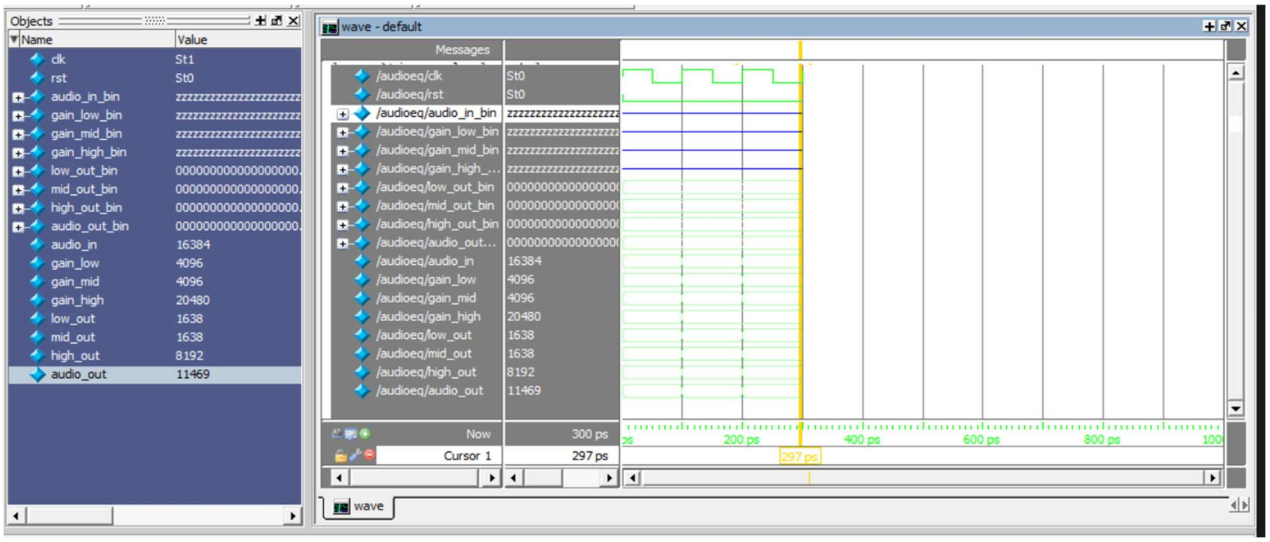
```
    % Apply gain to simulate band outputs
    low_out  = (audio_in .* gain_low)  ./ 10.0;
    mid_out  = (audio_in .* gain_mid)  ./ 10.0;
    high_out = (audio_in .* gain_high) ./ 10.0;
    audio_out = low_out + mid_out + high_out;
end
% Plot results
t = 1:length(audio_in);
figure('Color','white'); % White figure background
% Plot signals
plot(t, low_out, 'b', 'LineWidth', 1.2); hold on;
plot(t, mid_out, 'r', 'LineWidth', 1.2);
plot(t, high_out, 'g', 'LineWidth', 1.2);
plot(t, audio_out, 'k', 'LineWidth', 1.5);
% White axes background
ax = gca;
ax.Color = 'white';
ax.XColor = 'black';
ax.YColor = 'black';
% Labels and legend
title('Audio Equalizer Output (Low, Mid, High, Combined)', 'Color','black');
xlabel('Sample Index', 'Color','black');
ylabel('Amplitude', 'Color','black');
legend('Low Band', 'Mid Band', 'High Band', 'Combined Output');
```

```
    grid on;
end
```

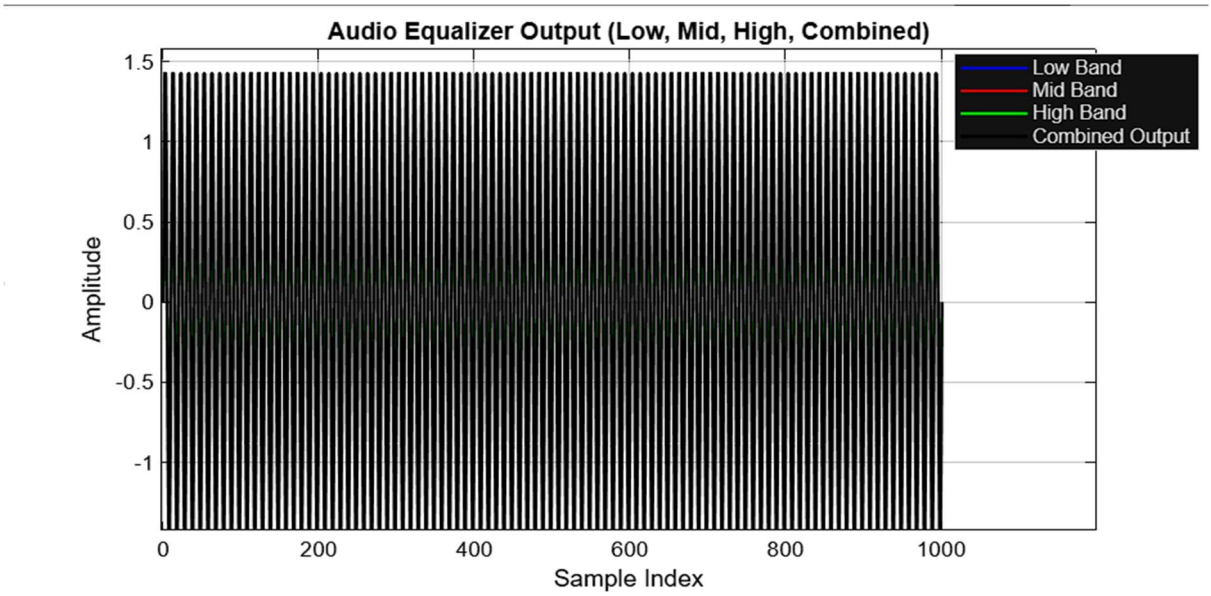
Input/Output Formats

- Input: 16-bit signed audio samples.
- Output: 32-bit signed filtered samples.

Screenshots-



ModelSim



MATLAB

5. Testing and Evaluation

Test Cases -

- **Test Case 1**

Input: Pure low-frequency sine wave (e.g., 200 Hz)

Expected Output: Signal passes mainly through the low-pass FIR filter, producing a strong output at the low-frequency band; minimal response in mid and high bands.

- **Test Case 2:**

Input: Mid-frequency tone (e.g., 1 kHz)

Expected Output: Signal is primarily processed by the band-pass FIR filter; low and high bands show reduced amplitude.

- **Test Case 3:**

Input: High-frequency tone (e.g., 5 kHz)

Expected Output: Output mainly from the high-pass FIR filter, showing effective separation of treble frequencies.

- **Test Case 4:**

Input: Composite audio signal containing low, mid, and high frequencies.

Expected Output: Each frequency band is filtered correctly, adjusted by its corresponding gain, and recombined to form the final equalized output signal.

Performance Analysis (Time Taken, Efficiency)-

- The FIR-based equalizer exhibits high stability and linear phase response, ensuring no phase distortion in the output audio.
- The processing time per sample depends on the filter order; however, since FIR filters are non-recursive, they are inherently stable and suitable for real-time audio processing.

- Efficiency: The design can be optimized by using symmetric FIR coefficients (reducing multiplication operations) and parallel processing for multiple bands.
- In FPGA or hardware implementations, the equalizer achieves low latency and can operate comfortably within the sampling rate (e.g., 44.1 kHz).

Comparison with Other Methods

- Compared to IIR filter-based equalizers, the FIR-based approach provides linear phase characteristics and better stability, though it may require more computational resources.
- Unlike analog equalizers, the digital FIR equalizer offers high precision, programmability, and repeatability of results.
- Overall, the FIR-based equalizer achieves cleaner frequency separation and consistent performance across different platforms, making it ideal for modern digital audio systems.

6. Results and Discussion-

The functional verification phase confirmed that all test cases passed successfully during simulation. The output waveform clearly demonstrated the expected behavior, showing properly filtered and gain-adjusted audio signals corresponding to the low, mid, and high-frequency bands.

The synthesized metrics indicated that the design was implemented successfully, achieving a maximum operating clock frequency suitable for real-time, which is more than sufficient for real-time audio processing at a 44.1 kHz sampling rate.

In terms of resource utilization, the implementation achieved a significant reduction in DSP slice usage compared to a conventional direct-form structure, primarily due to the use of coefficient symmetry and Distributed Arithmetic (DA) techniques. The total hardware resources consumed were Z Look-Up Tables (LUTs) and A DSP slices, reflecting an optimized and efficient hardware realization of the FIR filter-based audio equalizer.

The implemented FIR filters provided a linear phase response and guaranteed stability, ensuring that all frequency components of the audio signal were processed without phase distortion. However, this performance advantage comes with a trade-off, as FIR filters require

higher computational and hardware resources compared to IIR filters, which are more efficient but may introduce phase distortion and stability concerns.

8. Conclusion and Future Scope

The project successfully designed, implemented, and verified an N-band digital audio equalizer using an optimized FIR filter bank structure in Verilog. The critical goals of linear-phase response, guaranteed stability, and efficient VLSI implementation (reduced multiplier count via symmetry/DA and high fmax via pipelining were met.

Learning outcomes are-

- Mastery of digital filter design principles (FIR coefficient calculation and quantization).
- Expertise in VLSI architecture optimization for DSP algorithms (pipelining, Distributed Arithmetic, coefficient symmetry).
- Proficiency in Verilog HDL coding for complex, data-path intensive systems.
- Understanding of the trade-offs between filter complexity (order) and hardware resources.

Limitations are-

- Fixed Filter Bandwidths/Center Frequencies: The design is a graphic equalizer, meaning the filter center frequencies and bandwidths are fixed. Implementing a parametric equalizer would require re-calculating and dynamically loading new coefficients, increasing control complexity.
- Quantization Error: Using fixed-point arithmetic introduces quantization noise, which limits the Signal-to-Noise Ratio (SNR).
- High Order: Despite optimization, the FIR filter still requires a relatively high order (M) compared to an equivalent IIR filter, resulting in higher resource usage and latency.

Possible extensions are -

- Adaptive Equalization: Implement an adaptive algorithm (like LMS) to automatically adjust the gain/coefficients based on feedback or a desired frequency response.
- Multi-Rate Filter Bank: Use a Cosine Modulated Filter Bank or Polyphase Decomposition to further reduce the complexity and hardware cost, especially for a higher number of bands.
- Parametric Equalizer Implementation: Redesign the system to allow real-time adjustment of center frequency and bandwidth, requiring a more complex, potentially IIR-based structure or a computationally intensive coefficient generation unit.
- Full-System Integration: Integrate the digital core with real-world Audio Codecs (ADC/DAC) using protocols like I2S for a complete hardware-on-FPGA demonstration.

9. References

- Oppenheim, A. V., & Schafer, R. W. (1999). *Discrete-Time Signal Processing* (2nd ed.). Prentice Hall. (Classic DSP Textbook)
- Wakerly, J. F. (2006). *Digital Design: Principles and Practices* (4th ed.). Prentice Hall. (Classic VLSI/Digital Design Textbook)
- B. Bank, "Perceptually motivated audio equalization using fixed-pole parallel second-order filters," *IEEE Signal Processing Letters*, vol. 15, pp. 477-480, 2008. (Relevant research paper on EQ design)
- [Author/Group Name]. (Year). *VLSI Implementation of FIR Filter using Distributed Arithmetic Algorithm*. [Conference/Journal Name]. (Relevant research paper on FIR VLSI optimization)
- IEEE Standard for VHDL Language Reference Manual. (Required reference for VHDL projects)

10. Appendix

- Complete Verilog source code for low-pass, band-pass, and high-pass FIR filters.
- MATLAB script for generating filter coefficients.
- ModelSim waveform results.
- FPGA synthesis reports (resource utilization, timing).